Vibrations Workshop 7 - Dirac'n the Delta

By: Jake Peyser
ME301
3/26/2026

General Overview

In this assignment, the impulse *function* in conjunction with discontinuous forcing functions will be studied in order to acquire a superior intuition with respect to how mass spring damper systems respond to external disturbances. Experimental contributions will also be used in the study.

Section 1

In this section, a mass spring damper system with mass $m = 1$ kg, damping coefficient $R = 0.1$ kg/s and stiffness $k = 200$ N/m will be acted upon by various external forcing functions. In general, provided that f is a forcing functions and x is the position of the mass, the dynamics will be assumed to evolve according to the following IVP (Newton's 2nd Law):

$$m\ddot{x} + R\dot{x} + kx = f(t) \quad \text{s.t.} \quad x_0 = 0 \quad \text{and} \quad \dot{x}_0 = 0$$

Part (a)

The forcing function here is given by $f(t) = \delta(t - t_0)$, which has the Laplace transform of $F(s) = e^{-st_0}$. The response was determined in MATLAB using the lsim function, and is shown below:

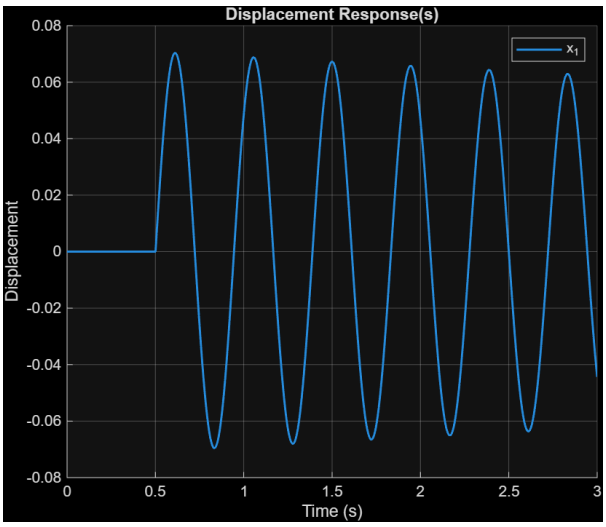

Figure 1: Displacement Response (a)

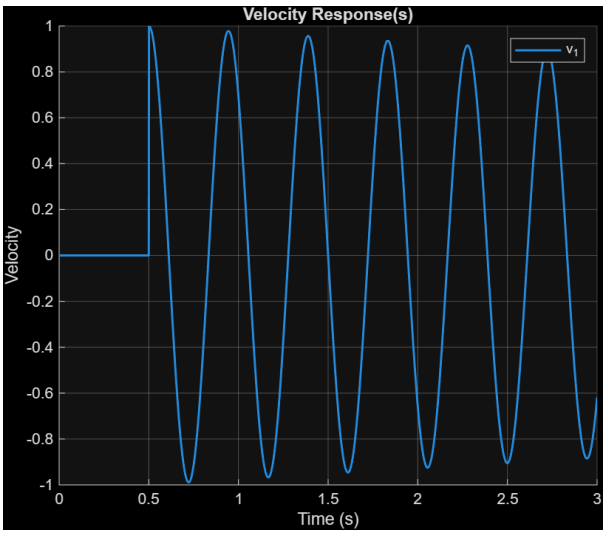

Figure 2: Velocity Response (a)

Observe that the system was momentarily driven (i.e. given an impulse) at $t = t_0 = 0.5$ s as expected, and then behaved as though is was driven afterwards given that it was given an initial positional/velocity offset.

Part (b)

The forcing function here is given by $f(t) = 0.2\delta(t) + 0.05\delta(t - t_0)$, which has the Laplace transform of $F(s) = 0.2 + 0.05e^{-st_0}$. The response was determined in MATLAB using the lsim function, and is shown below:

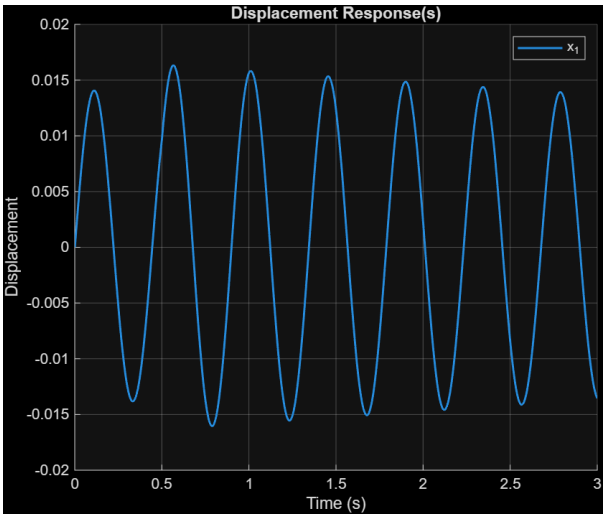

Figure 3: Displacement Response (b)

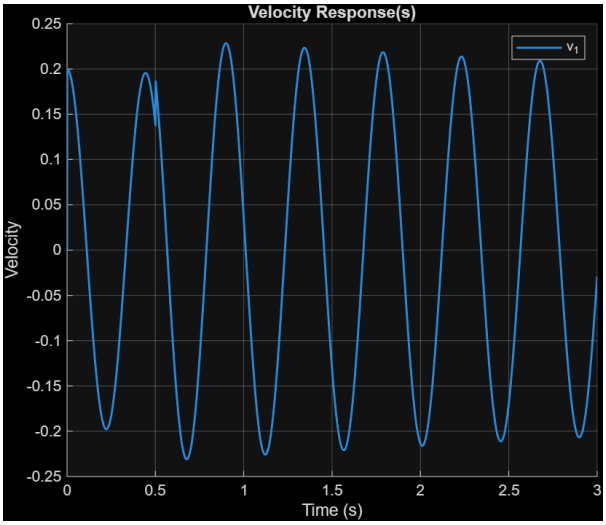

Figure 4: Velocity Response (b)

Observe that the system behaved as though it was given an initial velocity offset, and then was given an impulse at $t = t_0$ as expected (again). Cool stuff.

Part (c)

The forcing function here is given by $f(t) = u(t - t_0)$, which has the Laplace transform of $F(s) = e^{-st_0}/s$. The response was determined in MATLAB using the lsim function, and is shown below:

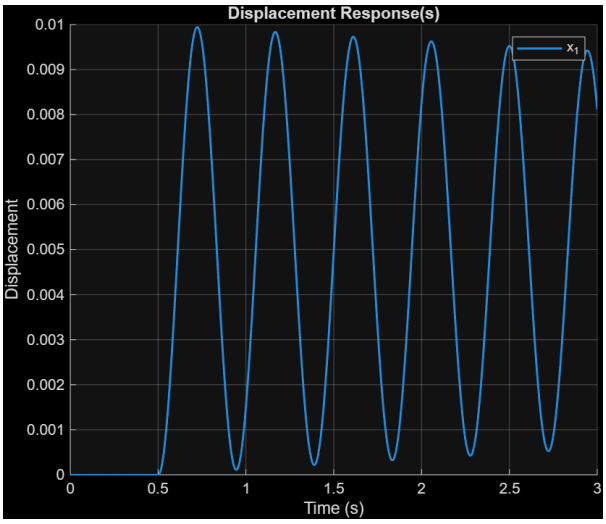

Figure 5: Displacement Response (c)

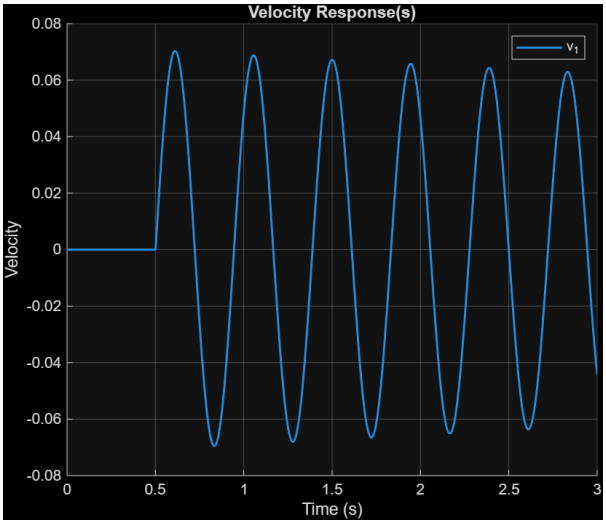

Figure 6: Velocity Response (c)

Observe that the system behaved as though it was totally unperturbed until $t = t_0$ from which it behaved as though a constant external force was applied to the system.

Part (d)

The forcing function here is given by $f(t) = e^{0.5t}u(t - t_0)$, which has the Laplace transform of $F(s) = e^{-(s-0.5)t_0}/(s - 0.5)$. The response was determined in MATLAB using the lsim function, and is shown below:

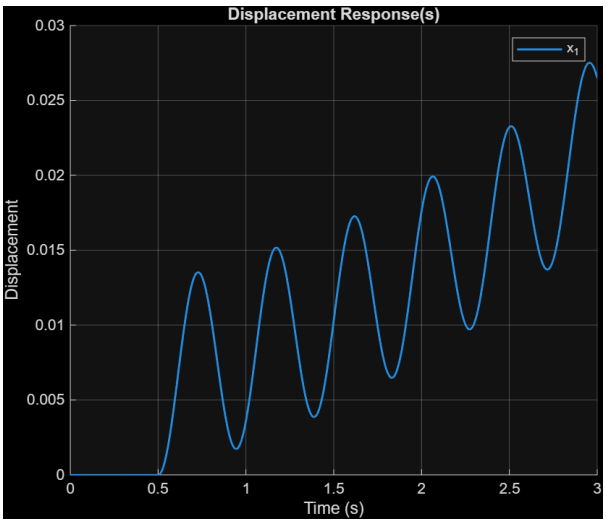

Figure 7: Displacement Response (d)

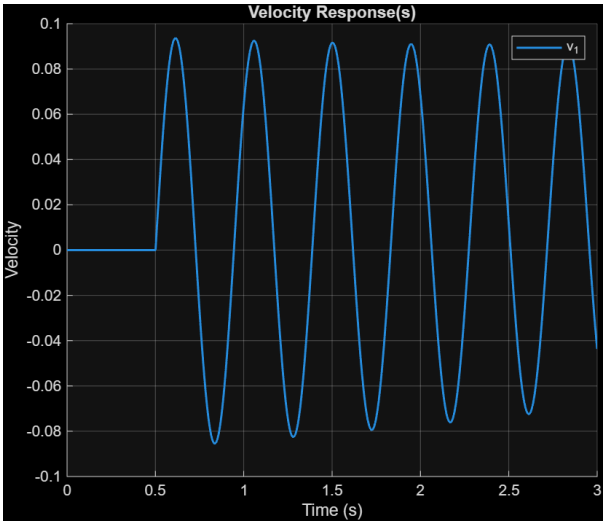

Figure 8: Velocity Response (d)

Observe that the system behaved as though it was totally unperturbed until $t = t_0$ from which it behaved as though an ever amplifying force was being applied to the system.

Part (e)

The forcing function here is given by:

$$f(t) = \begin{cases} t & \text{for} \quad 0 < t < t_0 \\ \sin(\omega_f t) & \text{for} \quad t \geq t_0 \end{cases}$$

which has the Laplace transform of:

$$F(s) = \int_0^{t_0} e^{-st} t \, dt + \int_{t_0}^{\infty} e^{-st} \sin(\omega_f t) \, dt = \frac{1 - e^{-st_0}(1 + st_0)}{s^2} + \frac{e^{-st_0}(s \sin(\omega_f t_0) + \omega_f \cos(\omega_f t))}{s^2 + \omega_f^2}$$

Because MATLAB's control system toolbox has issues with transcendental transfer functions, lsim could not be directly implemented to acquire the response of the system.

To remedy this, the system was simulated (via lsim) for $0 < t < t_0$, and then for $t_0 < t < 3$s with new *initial* conditions $x(t_0)$ and $\dot{x}(t_0)$ respectively. Both time intervals are equipped with their own forcing functions as prescribed. The temporal/response vectors were then combined for visual purposes. To be explicit, the respective (Laplaced) forcing functions for each time interval are given as such:

$$F_1(s) = \frac{1 - e^{-st_0}(1 + st_0)}{s^2} \quad \text{and} \quad F_2(s) = \frac{e^{-st_0}(s \sin(\omega_f t_0) + \omega_f \cos(\omega_f t))}{s^2 + \omega_f^2}$$

Also, for simplicity, let $\omega_f = 1$ rad/s. The respective responses are shown below:

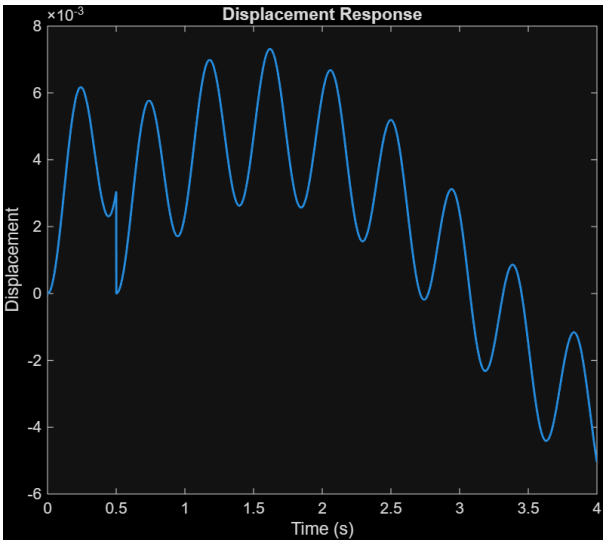

Figure 9: Displacement Response (e)

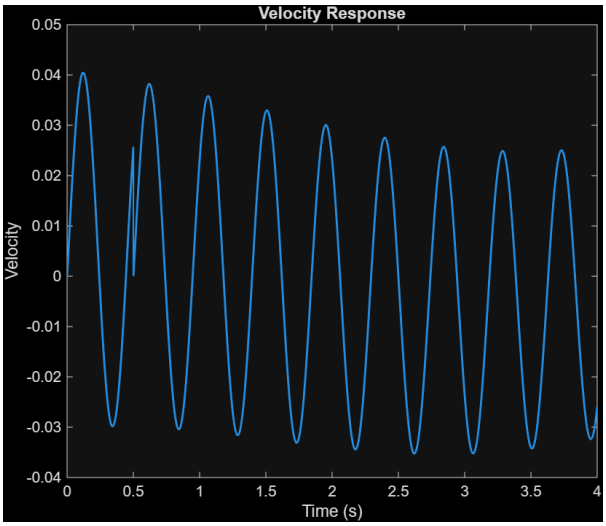

Figure 10: Velocity Response (e)

Not much can be said except this is behaving exactly as expected. The system undergoes a ramp force up until $t = t_0$, and then suddenly that ramp is replaced by a less prominent force, which we know to be a sinusoid. The kinematics are kind of interesting.

Section 2

In this section, the results from the impulse response lab will be shown an analyzed. To start, we could have a look at the accelerometer and force data (red hammer tip):

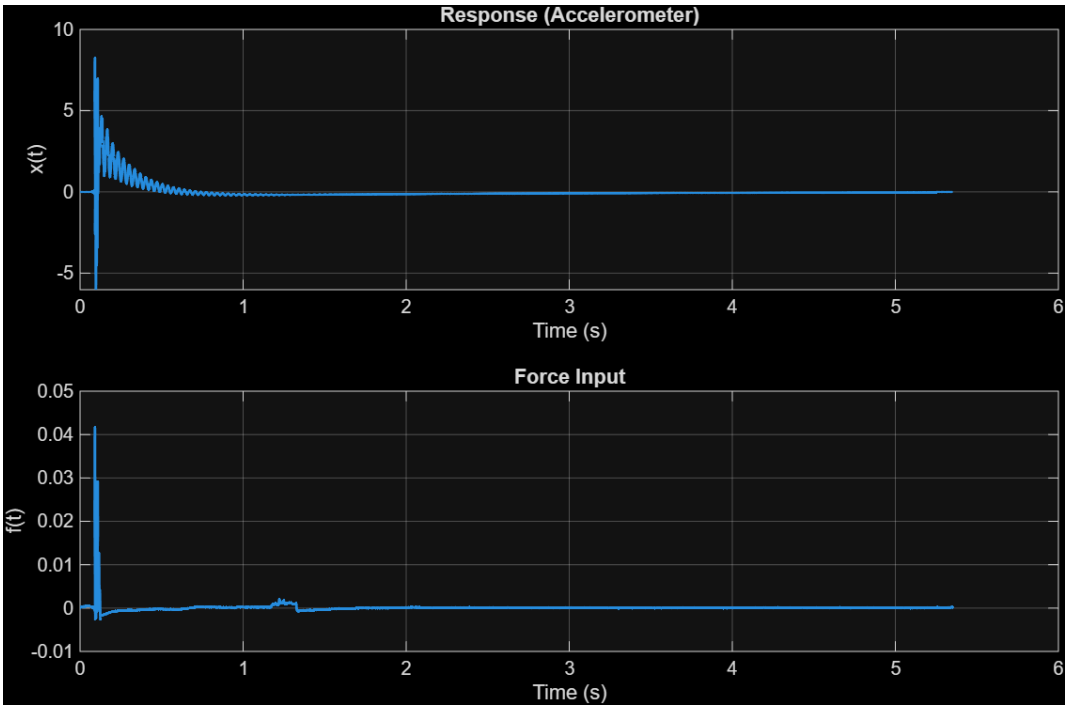

Figure 11: Accelerometer & Force Data (red hammer tip)

Unfortunately, there appear to be several impulse spikes (looks like three), which does not represent an ideal impulse disturbance function, but it's what was collected, and the other results were not too great either. With that out of the way, let's take their Fast Fourier Transforms!

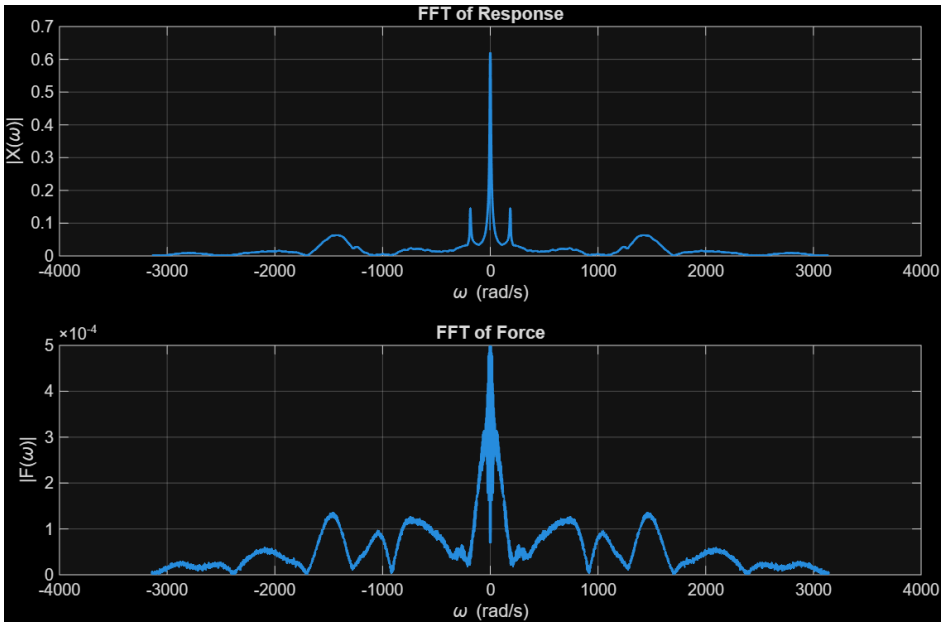

Figure 12: FFT of Accelerometer/Force Data

As expected, the FFTs look cool and all, but the FFT of the force signal is not constant at all, which is consistent with the failure to reproduce an impulse disturbance. With the FFTs worked out, the FRF (H) can be worked out as well:

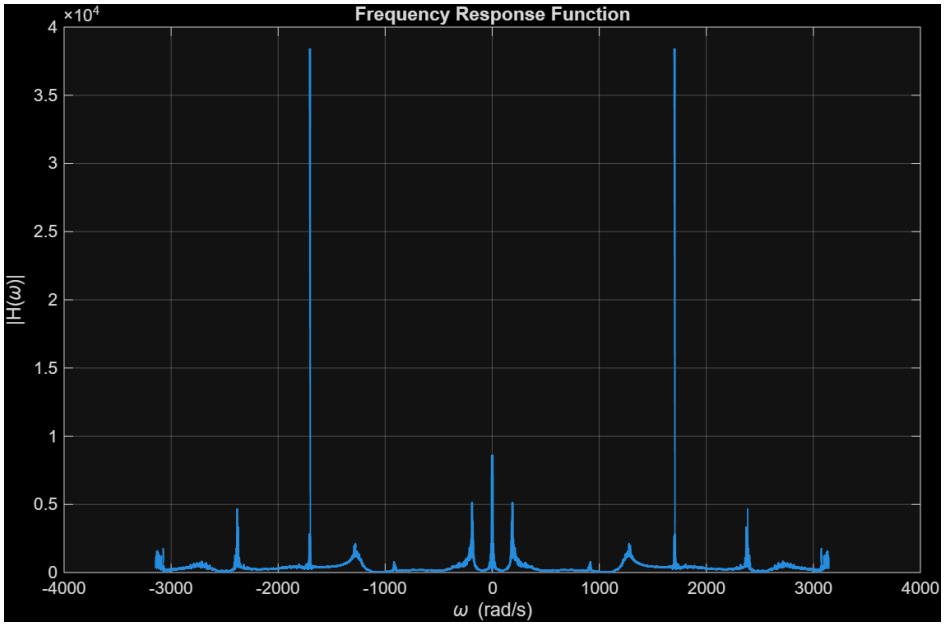

Figure 13: Frequency Response Function

The plot speaks for itself. In theory this should be *the* FRF for this system (regardless of the input).

The distinction between the FFTs of the measured signals and the FRF just above is that the former is the frequency analog of the signal input & output, and the latter is the frequency analog of the $\frac{\text{output}}{\text{input}}$. They are fundamentally different mathematical objects. One can note that when the input is very small (in the frequency domain), the FRF usually produces spikes at those locations. That is very evident here.

Next item on the menu is the impulse response. This is what the original accelerometer signal *in theory* should have looked like (shifted in time) if the forcing function were an ideal impulse disturbance, but never-mind that. Below is the plot of the impulse response function as a function of time:

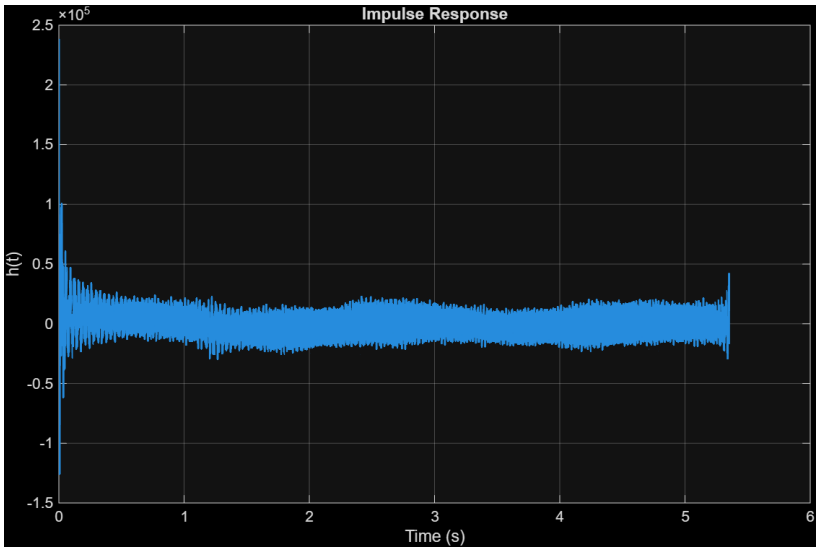

Figure 14: Impulse Response Function

So not the best looking IRF, but this is what MATLAB spit out; it's going to be taken as the word of god here, and so it won't be questioned. However, it must be said that the initial spike at $t = 0$ is totally nonphysical... In no way can such a large change in momentum be imparted on the system in such a small amount of time with the experimental methodology at hand. It is what it is though. These overall results can be compared to the M2 lab experiment:

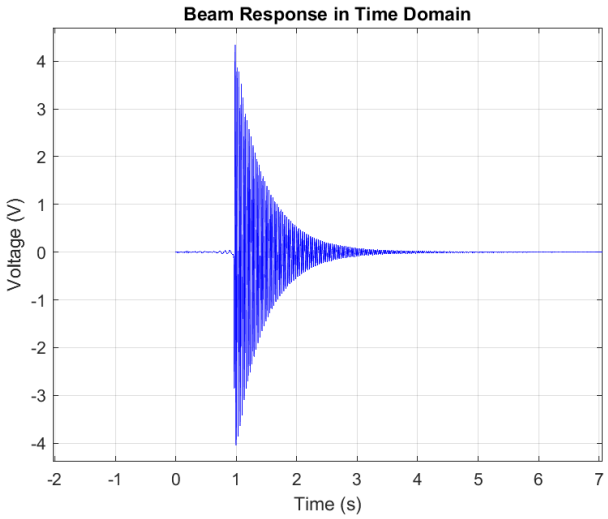

Figure 15: M2 Raw Data

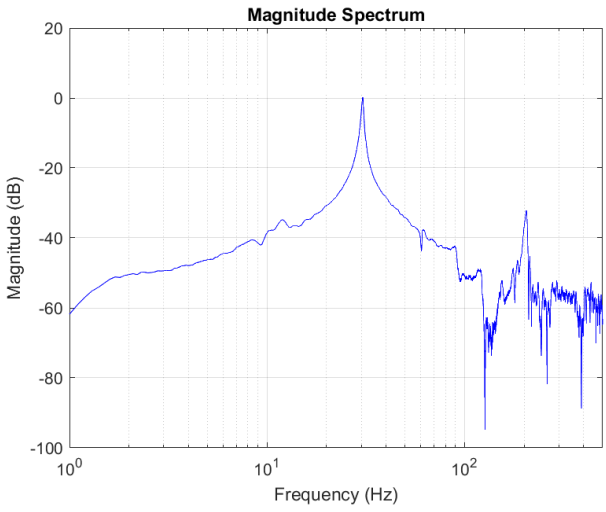

Figure 16: M2 FFT Applied to Raw Data

The way in which this data was collected involved displacing a cantilever following by letting the system oscillate in one smooth action. In this regard, the major peak in the FFT corresponds to the natural frequency of the system, and the other noise corresponds to the act of displacing the cantilever, and other random noise.

This is unlike the FFT examined before in this section. Specifically, the sharp spike in the middle corresponds to the offset caused by the impulses, whereas the adjacent spikes correspond to the natural frequency there. Moreover, the smoothed out area around 1500 rad/s appears to be from the successive impulses, but that may be an inaccurate observation. It should be pointed out that the natural frequency in the M2 FFT plot dominates, whereas the zero frequency dominates in the other FFT plot.

Conclusionary Remarks

This workshop built intuition for how mass spring damper systems react to sudden disturbances. The impulse response serves as the fundamental building block. In particular, an impulse delivers an instant kick, after which the system freely vibrates according to its natural dynamics.

Simulating piecewise forcing functions highlighted the limitations of direct Laplace-domain methods with lsim. Breaking up the problem into two durations did the trick though.

Experimentally, real hammer impulses were far from ideal, producing noisy FRFs and imperfect impulse responses. This underscored the gap between theory and practice. Overall, the exercises revealed both the power and the practical challenges of linear system dynamics.

MATLAB Code: Section 1: (a) - (d)

```
clc; clear; close all;

%% Parameters
m = 1;
R = 0.1;
k = 200;
T0 = 0.5;
w = 1;

%% Impedance Matrix
Z = @(s) ...
       [m*s+R+k/s;
        ];

%% Control System Toolbox Implementation
s = tf('s');
Z_tf = Z(s);

%% Laplaced External Force Vector & Initial Conditions
% F = [exp(-s*T0)]; %(a)
% F = [0.2+0.05*exp(-s*T0)]; %(b)
% F = [exp(-s*T0)/s]; %(c)
 F = [exp(-s*T0)*exp(0.5*T0)/(s-0.5)]; %(d)
% F = [(1-exp(-s*T0))*(1+s*T0)/(s^2) ...
%        +exp(-s*T0)*(s*sin(w*T0)+w*cos(w*T0))/(s^2+w^2)]; %(e)
x0 = [0];
v0 = [0];
```

```
28
29 %% Time Domain Parameters
30 t0 = 0;
31 t1 = 3;
32 dt = 0.001;
33
34 %% Coefficient Extraction
35 syms z
36 Z_sym = Z(z);
37
38 n = size(Z(1),1);
39 for i = 1:n
40     for j = 1:n
41         Zij = Z_sym(i,j);
42         M(i,j) = double(limit(Zij/z,z,+inf));
43         K(i,j) = double(limit(Zij*z,z,0));
44         R(i,j) = double(limit(Zij - M(i,j)*z - K(i,j)/z, z, 1));
45     end
46 end
47
48 % disp('M:'); disp(M);
49 % disp('R:'); disp(R);
50 % disp('K:'); disp(K);
51
52 %% Augmented Force Vector
53 F_aug = F + M*v0' - (K*x0')/s;
54
55 %% Dependent Laplace Variables
56 Y = inv(Z_tf);
57 V = Y*F_aug;
58 X = V/s + x0'/s;
59 H = Y/s;
60
61 %% Time-Domain
62 t = t0:dt:t1;
63 u = zeros(size(t));
64 u(1) = 1/(t(2)-t(1));
65 [v_resp, t_out] = lsim(V, u, t);
66 [x_resp, ~] = lsim(X, u, t);
67
68 % Extract signals
69 v = squeeze(v_resp(:, :, 1));
70 x = squeeze(x_resp(:, :, 1));
71
72 %% Plot Velocities
73 figure; hold on; grid on;
74 for k = 1:n
75     plot(t_out, v(:,k), 'LineWidth',1.4);
76 end
77 xlabel('Time (s)');
78 ylabel('Velocity');
79 title('Velocity Response(s)');
80 legend(arrayfun(@(k) sprintf('v_%d',k),1:n,'UniformOutput',false));
81
```

```
%% Plot Displacements
figure; hold on; grid on;
for k = 1:n
    plot(t_out, x(:,k), 'LineWidth',1.4);
end
xlabel('Time (s)');
ylabel('Displacement');
title('Displacement Response(s)');
legend(arrayfun(@(k) sprintf('x_%d',k),1:n,'UniformOutput',false));

%% BODE Plot (Y)
w = logspace(-2,2,1000);

figure; hold on; grid on;
legend_entries = {};

for i = 1:n
    for j = 1:n

        [mag,phase] = bode(Y(i,j), w);

        mag   = squeeze(mag);
        phase = squeeze(phase);

        subplot(2,1,1); hold on; grid on;
        semilogx(w,20*log10(mag),'LineWidth',1.4);
        ylabel('Modulus (dB)');
        title('Admittance Bode Plot');

        subplot(2,1,2); hold on; grid on;
        semilogx(w,phase,'LineWidth',1.4);
        ylabel('Phase (deg)');
        xlabel('\omega (rad/s)');

        legend_entries{end+1} = sprintf('Y_{%d%d}',i,j);
    end
end

subplot(2,1,1); legend(legend_entries,'Location','best');
subplot(2,1,2); legend(legend_entries,'Location','best');

%% BODE Plot (H)
figure; hold on; grid on;
legend_entries = {};

for i = 1:n
    for j = 1:n

        [mag,phase] = bode(H(i,j), w);

        mag   = squeeze(mag);
        phase = squeeze(phase);

        subplot(2,1,1); hold on; grid on;
```

```
136             semilogx(w,20*log10(mag),'LineWidth',1.4);
137             ylabel('Modulus (dB)');
138             title('Impulse Response Bode Plot');
139 
140             subplot(2,1,2); hold on; grid on;
141             semilogx(w,phase,'LineWidth',1.4);
142             ylabel('Phase (deg)');
143             xlabel('\omega (rad/s)');
144 
145             legend_entries{end+1} = sprintf('H_{%d%d}',i,j);
146         end
147     end
148 
149     subplot(2,1,1); legend(legend_entries,'Location','best');
150     subplot(2,1,2); legend(legend_entries,'Location','best');
```

MATLAB Code: Section 1: (e)

```
clc; clear; close all;

%% Parameters
m = 1;
R = 0.1;
k = 200;
T0 = 0.5;
w = 1;

%% Impedance
Z = @(s) [m*s + R + k/s];

s = tf('s');
Z_tf = Z(s);

%% Admittance
Y = inv(Z_tf);
H = Y/s;

%% Time parameters
t0 = 0;
t1 = 4;
dt = 0.001;
t = t0:dt:t1;

%% Laplace forcing pieces

F1 = (1 - exp(-s*T0))*(1 + s*T0)/(s^2);
F2 = exp(-s*T0)*(s*sin(w*T0) + w*cos(w*T0))/(s^2 + w^2);

%% Build "systems"
V1 = Y * F1;
X1 = V1 / s;

V2 = Y * F2;
X2 = V2 / s;

%% Approximate impulse input

u = zeros(size(t));
u(1) = 1/dt;    %   (t) approximation

%% Simulate with lsim

[v1, t_out] = lsim(V1, u, t);
[x1, ~]     = lsim(X1, u, t);

[v2, ~] = lsim(V2, u, t);
[x2, ~] = lsim(X2, u, t);

v1 = squeeze(v1);
x1 = squeeze(x1);
```

```
v2 = squeeze(v2);
x2 = squeeze(x2);

%% Truncate segments - I did use AI for this :D

idx1 = t_out <= T0;
idx2 = t_out >= T0;

t_seg1 = t_out(idx1);
t_seg2 = t_out(idx2);

v_seg1 = v1(idx1);
x_seg1 = x1(idx1);

v_seg2 = v2(idx2);
x_seg2 = x2(idx2);

%% Concatenate

t_full = [t_seg1; t_seg2(2:end)];
v_full = [v_seg1; v_seg2(2:end)];
x_full = [x_seg1; x_seg2(2:end)];

%% Plot Velocity

figure; grid on;
plot(t_full, v_full, 'LineWidth',1.4);
xlabel('Time (s)');
ylabel('Velocity');
title('Velocity Response');

%% Plot Displacement

figure; grid on;
plot(t_full, x_full, 'LineWidth',1.4);
xlabel('Time (s)');
ylabel('Displacement');
title('Displacement Response');

%% BODE (unchanged)

wvec = logspace(-2,2,1000);

figure; hold on; grid on;

[mag,phase] = bode(Y, wvec);
mag = squeeze(mag);
phase = squeeze(phase);

subplot(2,1,1);
semilogx(wvec,20*log10(mag),'LineWidth',1.4);
ylabel('Modulus (dB)');
title('Admittance Bode Plot');

```

```
107 subplot(2,1,2);
108 semilogx(wvec,phase,'LineWidth',1.4);
109 ylabel('Phase (deg)');
110 xlabel('\omega (rad/s)');
```

MATLAB Code: Section 2

```
clc; clear; close all;

%% Load Data
S = load("C:\Users\jacob\Downloads\Team B5 - Impact Hammer Measurements - Tip Red.mat");

A  = S.data;
f  = A(:,1);      % response (accelerometer)
x  = A(:,2);      % force input
fs = S.fs;

%% Time Domain Parameters
N  = length(x);
dt = 1/fs;
t  = (0:N-1)*dt;

%% FFTs
X = fft(x)*dt;
F = fft(f)*dt;

%% Frequency Response Function
H = X ./ F;

%% Impulse Response
h = ifft(H)/dt;

%% Centered FFTs
X_shift = fftshift(X);
F_shift = fftshift(F);
H_shift = fftshift(H);

%% Frequency axis
f_shift = (-floor(N/2):ceil(N/2)-1)*(fs/N);
w_shift = 2*pi*f_shift;

%% ---------- TIME DOMAIN ----------

figure;

subplot(2,1,1)
plot(t, x, 'LineWidth', 1.4)
grid on
xlabel('Time (s)')
ylabel('x(t)')
title('Response (Accelerometer)')

subplot(2,1,2)
plot(t, f, 'LineWidth', 1.4)
grid on
xlabel('Time (s)')
ylabel('f(t)')
title('Force Input')
```

```
52
53 %% ---------- FFTs ----------
54
55 figure;
56
57 subplot(2,1,1)
58 plot(w_shift, abs(X_shift), 'LineWidth', 1.4)
59 grid on
60 xlabel('\omega (rad/s)')
61 ylabel('|X(\omega)|')
62 title('FFT of Response')
63
64 subplot(2,1,2)
65 plot(w_shift, abs(F_shift), 'LineWidth', 1.4)
66 grid on
67 xlabel('\omega (rad/s)')
68 ylabel('|F(\omega)|')
69 title('FFT of Force')
70
71 %% ---------- FRF ----------
72
73 figure;
74
75 plot(w_shift, abs(H_shift), 'LineWidth', 1.4)
76 grid on
77 xlabel('\omega (rad/s)')
78 ylabel('|H(\omega)|')
79 title('Frequency Response Function')
80
81 %% ---------- IMPULSE RESPONSE ----------
82
83 figure;
84
85 plot(t, real(h), 'LineWidth', 1.4)
86 grid on
87 xlabel('Time (s)')
88 ylabel('h(t)')
89 title('Impulse Response')
```